

Failure (search fails mid-run) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (how a real deployment handles the messy edge cases). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

You need a checkpointer attached to your graph (from Doc09) for this exercise to mean anything — without one, there's no saved state to check after a crash.

The idea: make your search node raise an error on purpose (temporarily — comment it back out after). Run the graph. It should crash at that node. Then check: is whatever the graph had already done, before the crash, still there and inspectable?

Things to use:

- A checkpointer attached when compiling the graph (`checker=MemorySaver()` is enough for this test).
- A temporary `raise RuntimeError(...)` inside your search node.
- A `try/except` around your `graph.invoke(...)` call, since the graph will genuinely raise.
- `graph.get_state(config)` called afterward, using the same `thread_id`, to inspect what survived.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

LangGraph checkpoints state *between* node executions, not just at the very end. So if node A finished and wrote to state, then node B raises an exception, the checkpointer already has node A's write saved — you can inspect it with `graph.get_state(config)` even though the overall run failed.

```
# search_failure_practice.py
def search_node(state: dict) -> dict:
    raise RuntimeError("simulated vector store outage") # temporary, for this te
```

The exact pieces:

- `graph.get_state(config)` — returns a `StateSnapshot` with `.values` (the state as of the last successful checkpoint) and `.next` (which node was about to run — this tells you exactly where it stopped).
- **The `try/except` around `invoke()`** — the exception from your node genuinely propagates up through `invoke()`; this isn't something LangGraph silently swallows, so your test code needs to expect and catch it.
- **What "intact" means here, concretely** — any state fields written by nodes that ran successfully before the failure should still be present and correct in `get_state(config).values` after the crash.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Proving state survives a crash is only half the real problem. The other half: not every failure deserves the same response. A vector store that's down for 200 milliseconds because of a network blip is completely different from one that's down for an hour, or a query that's malformed and will fail identically every time you retry it — but a graph that just re-raises on any exception treats all three the same way: total failure, human has to intervene.

The real design question isn't just "does state survive a crash" — it's "which failures are worth retrying automatically, how many times, and how do you avoid retrying forever on something that will never succeed?"

That needs telling transient failures apart from permanent ones, and a retry with backoff instead of one immediate attempt:

The extra pieces needed:

- A small set of exception types you treat as **transient** (worth retrying) — a timeout, a connection error — versus everything else, treated as **permanent** (fail immediately, no point retrying a malformed query five times).
- A retry loop with a max attempt count and a short delay that grows between attempts (`time.sleep(2 ** attempt)`), so a single blip gets absorbed but a genuinely down service doesn't get hammered forever.
- A decision about what happens after retries are exhausted — for this document's graph, that's exactly where `search_failure`'s original lesson applies: the node still raises (or routes to a "search unavailable" branch), and whatever ran before it is still safely checkpointed either way.

Sketch the retry loop's shape — what it catches, how many times, how long it waits — before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both prove the same true thing — a crash doesn't erase earlier progress — by making the node fail exactly once and checking what's left. Advanced asks what a real deployment does *before* giving up: retry a transient failure a few times with growing delays, but recognize a permanent failure immediately instead of wasting time repeating it — and only after that's exhausted does the "state survives the crash" guarantee from Basic and Intermediate actually get used, as the safety net underneath the retry logic, not a replacement for it.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
temporarily change search_node to always raise an error

compile the graph with a checkpointer, using a config with a thread_id

try:
    run the graph
except the error:
    print "caught it, as expected"

check the saved state:
    call get_state with the same config
    print what's in it -- confirm earlier steps' data is still there
```

Here's almost the whole thing — just try running it and reading it line by line:

```
# search_failure_practice.py
from langgraph.checkpoint.memory import MemorySaver

def search_node(state):
    raise RuntimeError("simulated outage")    # temporary

checkpointer = MemorySaver()
graph = builder.compile(checkpointer=checkpointer)

config = {"configurable": {"thread_id": "failure-test-1"}}

try:
    graph.invoke({"task": "something needing search"}, config)
except RuntimeError as e:
    print("Caught:", e)

snapshot = graph.get_state(config)
print(snapshot.values)
```

Expected output: `Caught: simulated outage` , followed by whatever state fields were written before `search_node` ran — the original `task` , at minimum. Remember to revert the temporary `raise` afterward.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
temporarily:
    def search_node(state: dict) -> dict:
        raise RuntimeError("simulated vector store outage")

config = {"configurable": {"thread_id": "failure-test-1"}}

try:
    graph.invoke({"task": "something needing search"}, config)
except RuntimeError as e:
    print(f"Caught expected failure: {e}")

snapshot = graph.get_state(config)
print("State that survived:", snapshot.values)
print("Was about to run:", snapshot.next)

# assert whatever ran before search_node wrote correctly, e.g.:
assert "task" in snapshot.values
```

```
# search_failure_practice.py
config: dict = {"configurable": {"thread_id": "failure-test-1"}}

try:
    graph.invoke({"task": "something needing search"}, config)
except RuntimeError as e:
    print(f"Caught expected failure: {e}")

snapshot = graph.get_state(config)
```

Add the assertions confirming the surviving state is correct, print `snapshot.next`, and remember to revert the temporary `raise`, then compare against the [Solution](#).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```

TRANSIENT_ERRORS = (TimeoutError, ConnectionError)
MAX_ATTEMPTS = 3

function search_node(state):
    for attempt in range(MAX_ATTEMPTS):
        try:
            return call retrieve(), save into sources
        except a TRANSIENT_ERRORS exception:
            if this was the last attempt: raise it for real
            wait (attempt+1) seconds, then try again
        except anything else:
            raise immediately -- no point retrying a permanent failure

test:
    make retrieve() raise TimeoutError twice then succeed -> confirm it recovers
    make retrieve() raise a permanent error -> confirm it fails on the first try,

```

Here's almost the whole thing — fill in the missing piece yourself:

```

# search_failure_practice.py
import time

TRANSIENT_ERRORS = (TimeoutError, ConnectionError)
MAX_ATTEMPTS = 3

def search_node(state: dict) -> dict:
    for attempt in range(MAX_ATTEMPTS):
        try:
            results = retrieve(state["task"])
            return {"sources": results}
        except TRANSIENT_ERRORS as exc:
            # your turn: if this was the last allowed attempt, re-raise exc
            # so the caller (and the checkpointer's "state survives a crash"
            # guarantee from Hint 1) still applies; otherwise sleep briefly
            # and let the loop try again
            ...

```

```
# a non-transient exception is not caught here at all --  
# it propagates immediately, on the first attempt
```

Fill in the retry-exhaustion check and the backoff sleep yourself, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both make the node fail exactly once, on purpose, and confirm the checkpointer's promise: whatever ran before is still there. Advanced puts a retry loop *in front of* that same node, so a transient failure gets a few chances to resolve itself before the graph gives up at all — and only failures that are either permanent, or that outlast every retry attempt, ever reach the point Basic and Intermediate were testing. The checkpointing guarantee doesn't change; Advanced just makes sure you're not relying on it for failures that a simple retry could have absorbed.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)